



Institución
Universitaria
Reacreditada en Alta Calidad

MANUAL DE USUARIO

David Jiménez Murillo
Leonardo Duque Muñoz
Andrés Eduardo Castro Ospina

GENERADOR AUTOMÁTICO DE DATASETS DE PYTORCH

Índice

1. Introducción a Datasets y Dataloaders en Pytorch	2
1.1. Datasets preexistentes en Pytorch	2
1.2. Cargar un dataset preexistente	2
2. Generador Automático de Datasets de Pytorch	3
3. Prerrequisitos	3
4. Guía de Usuario	3
4.1. Organización de la base de datos	4
4.2. Ejecución del programa	4
4.2.1. Especificación de la ubicación de los datos	4
4.2.2. Cambio de resolución de las imágenes	5
4.2.3. Formatos de imagen soportados	6
4.2.4. Transformadas de torchvision	6
4.2.5. Generación de código fuente	7
4.3. Ejemplo de uso	8
4.4. Cómo importar la clase en el código fuente	9

1. Introducción a *Datasets* y *Dataloaders* en *Pytorch*

La idea detrás de los *datasets* en *Pytorch* es desacoplar el código para cargar y preprocesar los datos de un *dataset* del código de entrenamiento de un modelo con el fin de facilitar el mantenimiento y legibilidad del código. *Pytorch* dispone de dos clases para este propósito:

- `torch.utils.data.DataLoader`
- `torch.utils.data.Dataset`

que permiten cargar y usar los *datasets* preexistentes en la librería *torchvision* así como bases de datos propias. La clase *Dataset* almacena los datos y etiquetas en memoria, mientras que la clase *Dataloader* crea un iterable alrededor de una instancia cualquiera de la clase *Dataset* para acceder a las muestras.

1.1. *Datasets* preexistentes en *Pytorch*

Pytorch cuenta con algunas de las bases de datos más comunes en aprendizaje de máquinas. Entre los tipos de *datasets* preexistentes en *Pytorch* se encuentran: [imágenes](#), [texto](#), y [audio](#).

1.2. Cargar un *dataset* preexistente

Los *datasets* preexistentes de *pytorch* se encuentran en las librerías *torchvision*, *torchtext* y *torchaudio*. Aunque para cargar cada *dataset* preexistente en las librerías se tienen diferentes parámetros, la función de cada uno de ellos en general viene siendo similar. Por ejemplo, para cargar la base de datos “Fashion-MNIST” de acuerdo a la [documentación oficial](#), se tiene:

```
import torch
from torch.utils.data import Dataset
from torchvision import datasets
from torchvision.transforms import ToTensor

training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)

test_data = datasets.FashionMNIST(
    root="data",
    train=False,
    download=True,
    transform=ToTensor()
)
```

En este código fuente se crea instancia que hereda de la clase *dataset*, dónde:

- “root” es la ruta donde se encuentran los datos almacenados
- “train” especifica si se va a usar la partición de entrenamiento o de pruebas
- “download” especifica si se quiere descargar el dataset o no

- “transform” y “target_transform” especifican las transformaciones que se quieren hacer sobre los datos y las etiquetas

2. Generador Automático de *Datasets* de *Pytorch*

Este generador es una herramienta diseñada con el fin de facilitar el proceso de carga de datos para algoritmos de aprendizaje de máquina, escritos usando la librería *Pytorch*, cuando se tienen bases de datos propias. A continuación damos un guía de su funcionamiento.

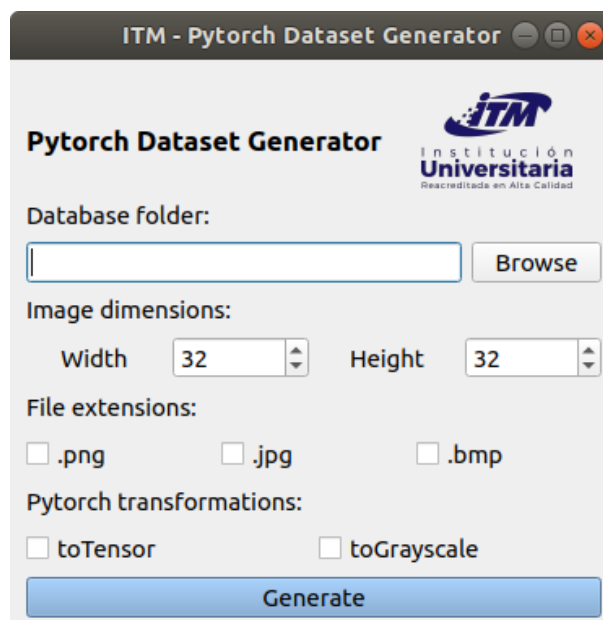
3. Prerrequisitos

-  **Operating system:**  Windows,  Linux
-  **Lenguaje de programación:**  Python >3.6
-  Cualquier editor de texto.
-  Un medio de almacenamiento para guardar el archivo generado.

4. Guía de Usuario

El generador automático consta de una sencilla interfaz gráfica desde la cual el usuario puede establecer los parámetros que requiere implementar para carga y pre-procesamiento de datos por medio de las diferentes funcionalidades del programa, usando botones y listas de chequeo. La interfaz gráfica de usuario se muestra en la Figura 1:

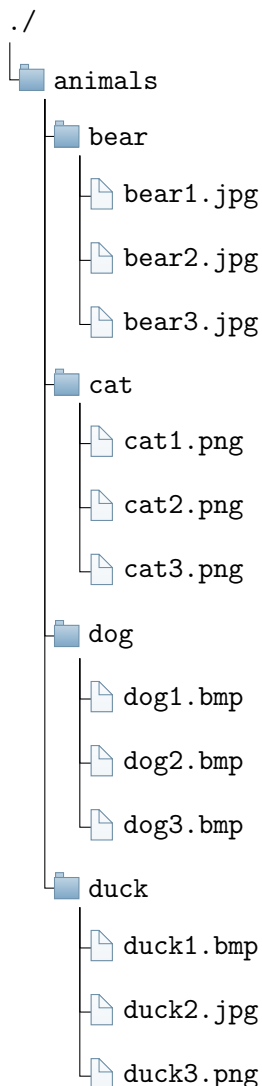
Figura 1: Interfaz gráfica de usuario



4.1. Organización de la base de datos

Previamente a usar el programa para generar el código fuente, se deben organizar los datos de tal forma que haya una carpeta principal y dentro de esta estén las imágenes separadas por clase dentro de diferentes subcarpetas. Cada subcarpeta aloja las imágenes de una determinada clase y debe tener el nombre de dicha clase.

En la figura se muestra la estructura en árbol que debe tener el directorio. En este caso se tendría la base de datos “animals”, la cual, según la distribución de directorios, tiene las clases “bear”, “cat”, “dog” y “duck”; y dentro de la carpeta de cada clase están incluidos los archivos de imagen.



4.2. Ejecución del programa

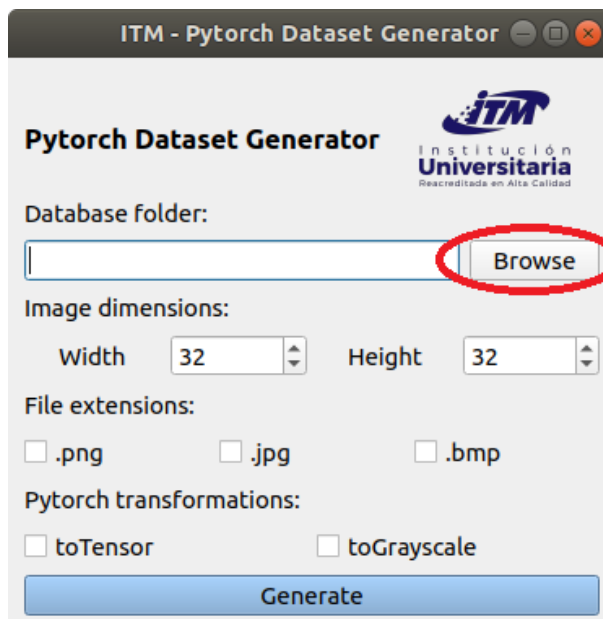
Luego de tener organizada la base de datos, el usuario debe dirigirse a la carpeta donde se encuentra el archivo ejecutable y hacer doble clic en este para abrir el programa. Se desplegará la interfaz gráfica de usuario del programa y desde allí ya podrá especificar los parámetros que requiera para los datos de su algoritmo de aprendizaje.

4.2.1. Especificación de la ubicación de los datos

En primer lugar se debe buscar la carpeta donde se encuentran los datos, para ello se debe hacer clic en el botón *Browse* para abrir el navegador de carpetas y seleccionar la indicada. Una vez seleccionada la carpeta, aparecerá su ruta en el cuadro de texto

a la izquierda del botón, también puede introducirse manualmente la ruta por medio del teclado. En la Figura 2 se señala la ubicación del botón *Browse*:

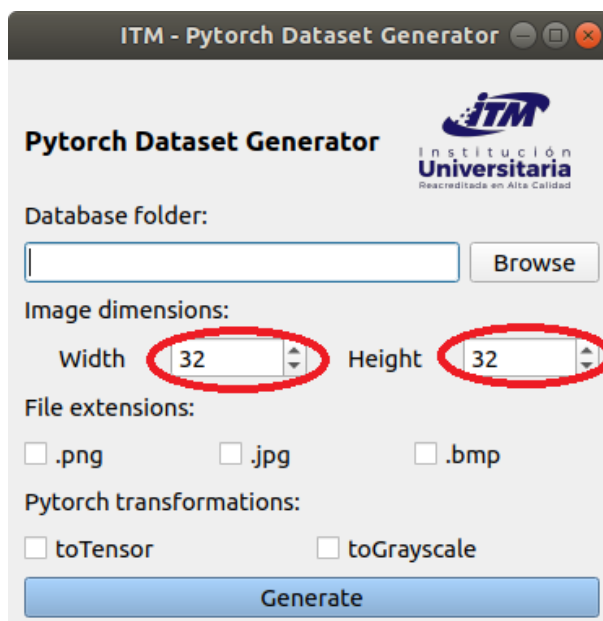
Figura 2: Ubicación del botón *Browse* en la interfaz



4.2.2. Cambio de resolución de las imágenes

Luego se debe indicar al programa la dimensión o resolución en píxeles a la que se desea cargar las imágenes para alimentar el algoritmo de aprendizaje. La dimensión mínima permitida es 8×8 , y la máxima dimensión permitida es 256×256 . Por defecto, se encuentra establecido con dimensiones de 32×32 como se muestra en la Figura 3. Por teclado puede asignarse cualquier tamaño o por medio de los botones incrementar o decrementar este valor de uno en uno.

Figura 3: Cuadro de texto y botones para establecer las dimensiones a las que se desea cargar las imágenes



4.2.3. Formatos de imagen soportados

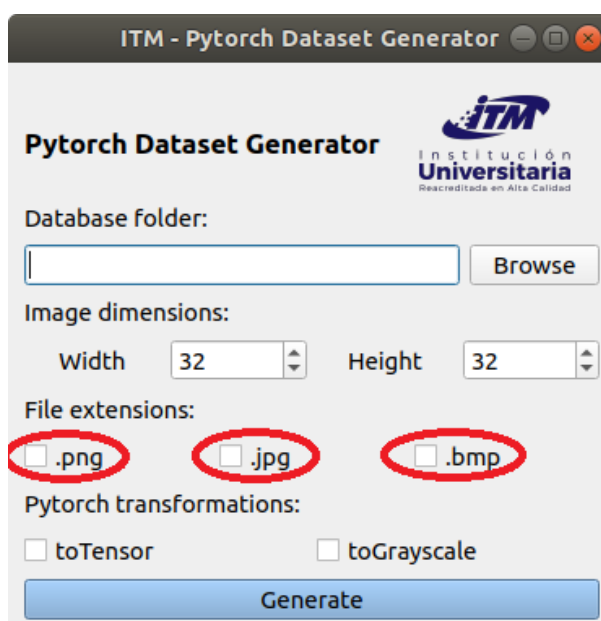
Después de especificar el tamaño debe indicarse los formatos de imagen con los que se desea trabajar. Los formatos de imagen soportados son aquellos que tienen como extensión:

- *jpg*
- *png*
- *bmp*

En la Figura 4 se muestra la ubicación de las casillas de selección de extensión. Pueden elegirse varios formatos de imagen al tiempo, para ello basta con señalar la casilla correspondiente al formato que se desea cargar.

El programa cargará solo los archivos que tengan una de estas extensiones, si existen otros archivos con extensión diferente a las que fueron marcadas en las casillas de chequeo, serán ignorados por el programa.

Figura 4: Extensiones de archivo soportadas



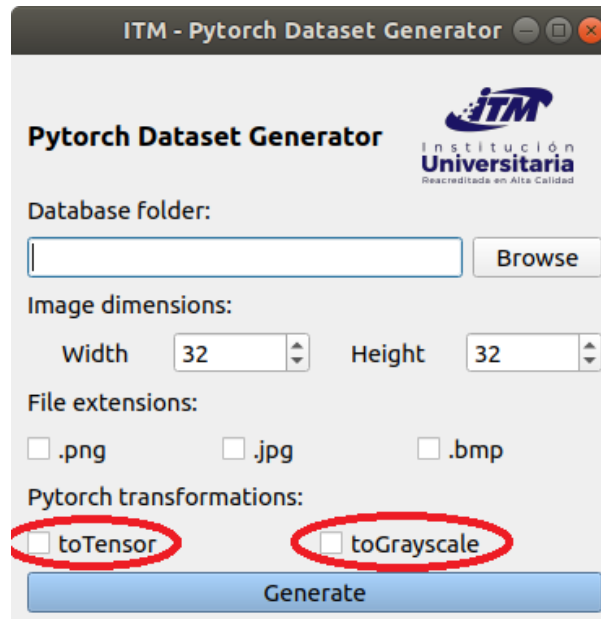
4.2.4. Transformadas de torchvision

Las transformadas de *torchvision* son opciones de preprocesamiento de datos que ya vienen incluidas por defecto en la librería *torchvision*. El programa incluye dos opciones de transformadas que son:

- “toTensor” la cual convierte el arreglo de la imagen, que generalmente se carga a un arreglo de “numpy”, a tensor, que es el tipo de dato que soporta *Pytorch* para realizar cálculos.
- “toGrayscale” convierte las imágenes a escala de grises, reduciendo el número de canales de la imagen, que en el caso de una imagen RGB, pasaría de tener tres canales a tener un solo canal.

Ambas transformadas son muy comunes en el preprocesamiento de datos para alimentar algoritmos de aprendizaje de máquinas. En la Figura 5 se señalan las transformadas de *torchvision* que se desea incluir en el *dataloader*, las cuales pueden seleccionarse marcando la casilla de chequeo de la transformada requerida.

Figura 5: Transformadas de *torchvision*

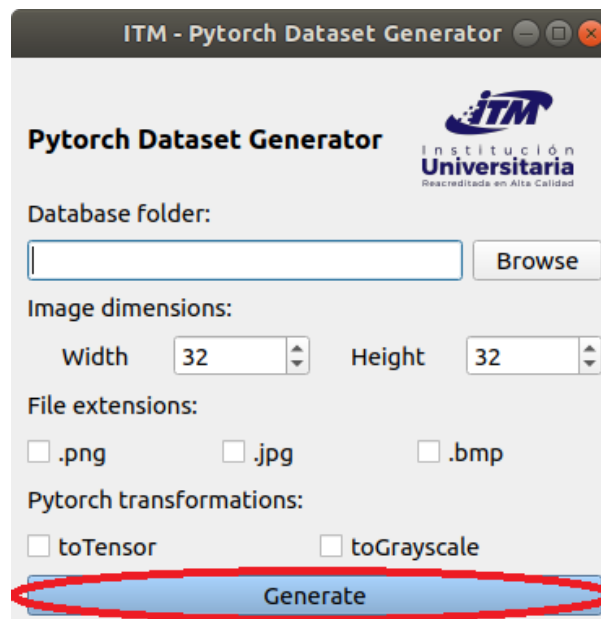


4.2.5. Generación de código fuente

Por último, haciendo clic en el botón *Generate* como se muestra en la Figura 6, se creará un archivo de código fuente en *Python* llamado "dl.py"; este archivo contiene la clase "*ImageDataset*" que hereda de la clase *Dataset* de *Pytorch*, y cuyos parámetros se sobre escriben de acuerdo con la configuración establecida por el usuario en los pasos anteriores.

El archivo con el código fuente se genera en el mismo directorio donde se encuentra ubicado el archivo ejecutable del programa. La clase *dataset* contenida en el archivo está lista para ser importada en el código de entrenamiento del algoritmo de aprendizaje.

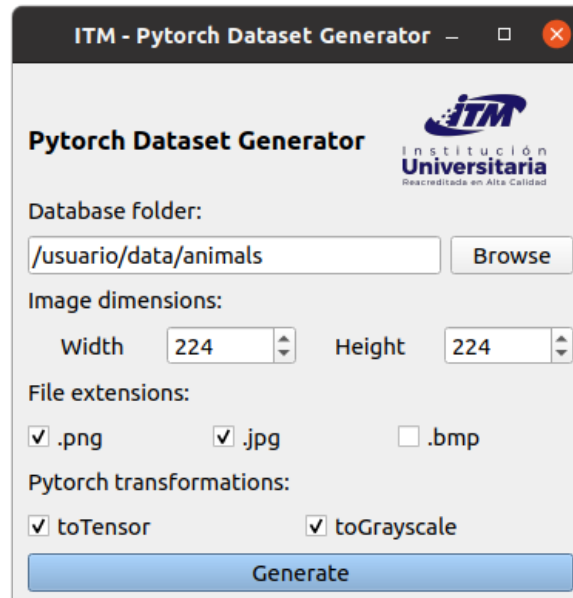
Figura 6: Ubicación del botón *Generate*



4.3. Ejemplo de uso

En la siguiente Figura 7 se tiene un ejemplo de uso del programa:

Figura 7: Ejemplo de uso de la aplicación



En este ejemplo los parámetros elegidos fueron los siguientes:

- Ubicación de la base de datos: /usuario/data/animals
- Dimensiones de la imagen: 224 × 224
- Extensiones de archivo: .png y .jpg
- Transformadas: toTensor y toGrayscale

Con estos parámetros la aplicación genera un archivo dl.py en el directorio de trabajo con el siguiente código fuente:

```
import os
import pandas as pd
import torch
from torch.utils.data import Dataset
from torchvision import transforms
from PIL import Image
from PIL import ImageFile
ImageFile.LOAD_TRUNCATED_IMAGES = True

class ImageDataset(Dataset):

    def __init__(self, target_transform=None):

        self.path = '/usuario/data/animals'
        self.transform = transforms.Compose([transforms.Grayscale(), \
        transforms.ToTensor()])
        self.target_transform = target_transform
```



```

self.dims = (224, 224)
self.ext = [ 'png', 'jpg' ]

subdirs = next(os.walk(self.path))[1]

L = []
for i, class_ in enumerate(subdirs):
    imgs = next(os.walk(os.path.join(self.path, class_)))[2]

    for img in imgs:
        if img[-3:] in self.ext:
            pathimg = os.path.join(self.path, class_, img)
            L.append([pathimg, i])

self.df = pd.DataFrame(L, columns=[ 'Path', 'Class' ])

def __len__(self):

    return self.df.shape[0]

def __getitem__(self, idx):

    img_path = self.df.at[idx, 'Path']
    label = self.df.at[idx, 'Class']

    image = Image.open(img_path)
    image = image.resize(self.dims)

    if self.transform:
        image = self.transform(image)
    if self.target_transform:
        label = self.target_transform(label)

    return image, label

```

4.4. Cómo importar la clase en el código fuente

El proceso de carga de la clase *ImageDataset* es bastante sencillo, tan solo se necesita importar el archivo generado desde el código de entrenamiento, con esta línea de código:

```
from dl import ImageDataset # Importa la clase generada
```

Luego debe crearse una instancia de la clase *ImageDataset*, que es la que produce el código fuente, y llevarla como argumento a una nueva instancia de la clase *ImageDataset* de *Pytorch*, especificando el tamaño de *batch* y otros parámetros propios de esa clase como el parámetro *shuffle* que le indica al programa que se desea cargar los datos aleatoriamente. En el siguiente fragmento de código se muestra con un ejemplo como es la estructura completa del código de carga:

```

from torch.utils.data import DataLoader
from dl import ImageDataset

animales = ImageDataset()
animales_ds = DataLoader(animales, batch_size=2, shuffle=True)
images, labels = next(iter(animales_ds))

```

La última línea de código produce un iterador sobre la base de datos que lleva las imágenes del *batch* a la variable *images* y las etiquetas las lleva a la variable *labels*. En la Figura 8 tenemos el resultado graficando para un *batch* de cuatro imágenes con sus respectivas etiquetas:

Figura 8: Ejemplo de uso de la aplicación con un batch de cuatro imágenes

